

C++20 Coroutines Demystified: Build a Generator From Scratch

A beginner-friendly guide to the coroutine mental model, `promise_type`, and building your first custom generator.

13 min read · Mar 26, 2026



Sagar

Following ▾



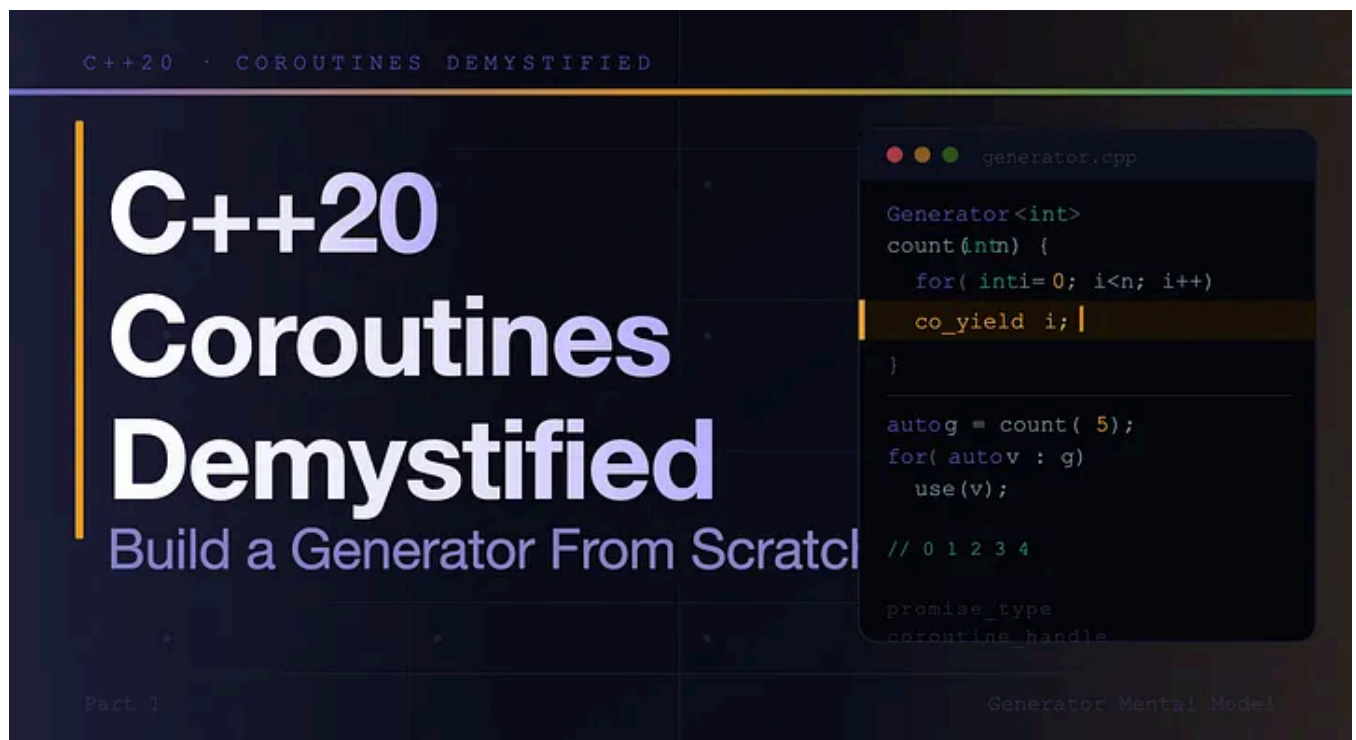
Listen



Share



More



A Confession Before We Dive In

I wanted to write about C++20 coroutines for years, but I was *terrified*.

This is one of those topics you can understand deeply in your head — get generators working, Tasks firing, symmetric transfers humming — but when someone asks

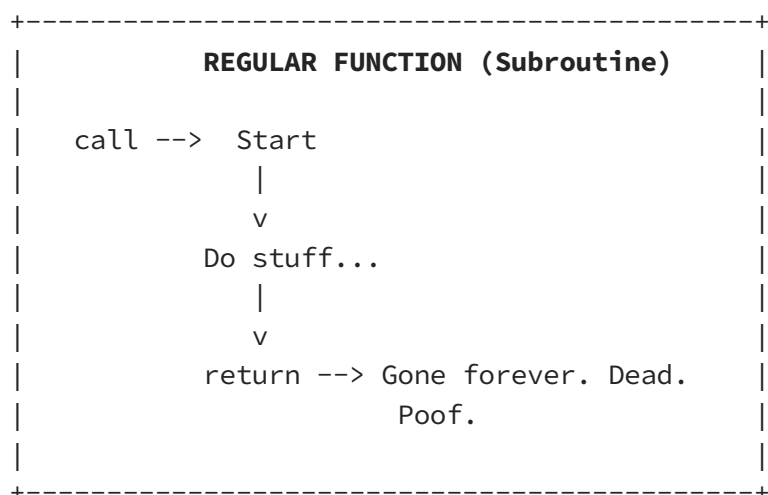
“Wait, how does it *actually* work?”, your brain freezes. The mental model exists, but explaining it clearly? That’s a whole different beast.

Then I realized: what’s the harm in trying?

This tutorial is my best attempt to explain coroutines with crystal-clear visual diagrams and working code. If readers get it, great! If not, your feedback will help me iterate and improve. Let’s crack this together.

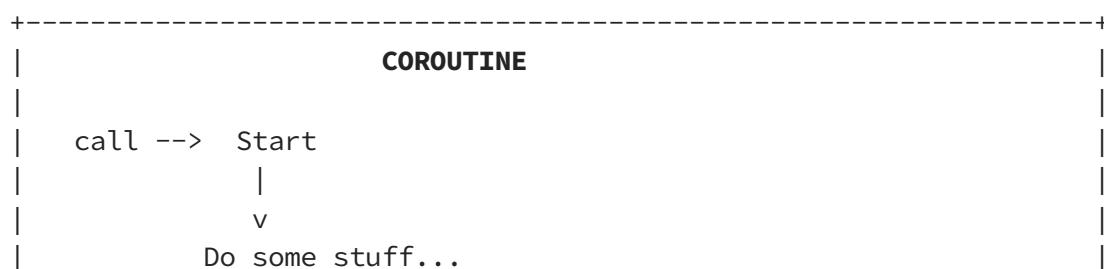
First Things First: What Even IS a Coroutine ?

You already know regular functions (a.k.a. subroutines). They’re simple folk:



A regular function is like a one-night stand. It comes in, does its thing, and leaves. No callbacks. No texts. Nothing.

A coroutine is different:



```

      |
      |
      |  v
      |  SUSPEND --> "I'll be back. Hold my state."
      |
      |  .
      |  . (caller does other things, lives life)
      |  .
      |  RESUME --> "Miss me?"
      |
      |  |
      |  |  v
      |  |  Do more stuff...
      |  |
      |  |  v
      |  |  SUSPEND again (or return/co_return)
      |  |
      |  |  .
      |  |  . (this can happen many times!)
      |  |  .
      |  |  RESUME --> "Back again."
      |  |
      |  |  v
      |  |  co_return --> Done for real this time.
      |
  
```

A coroutine can suspend its execution and resume later, picking up right where it left off — with all its local variables intact.

That's it. That's the whole idea. Everything else is just machinery.

And oh boy, is there a lot of machinery.

The Three Magic Words

In C++20, a function becomes a coroutine if (and only if) its body contains one of these three keywords:

```

+-----+
|
|  co_await  --> "Pause me. I'll wait for something."
|
|  co_yield  --> "Here's a value. Now pause me."
|              (It's basically co_await in a trench coat)
|
|  co_return --> "I'm done. Here's my final answer."
|
  
```



You do **not** mark a function with any special keyword like `coroutine` or `async`. The compiler sees `co_await`, `co_yield`, or `co_return` in the body and goes:

Oh. OH. This is a coroutine. Time to generate a MOUNTAIN of boilerplate.

The Big Picture: Who Writes What ?

This is where most tutorials lose people. So let’s get crystal clear about the division of labor:

WHO DOES WHAT?	
YOU (The Developer)	COMPILER (The Machine)
1. Write the coroutine body (the fun part)	1. Allocates coroutine frame (heap, usually)
2. Define the RETURN TYPE (e.g., <code>Generator<int></code>)	2. Copies parameters into the frame
3. Define the PROMISE TYPE (the control center)	3. Constructs the promise object inside the frame
4. Optionally define custom AWAITABLE types	4. Calls promise methods at specific points
	5. Manages suspend/resume via <code>coroutine_handle</code>
	6. Destroys everything when done

Think of it like a restaurant:

- **You** are the chef (you write the recipe — the coroutine body)
- The **promise type** is the kitchen manager (it decides how things flow)

[Open in app ↗](#)

The Coroutine Frame: Where Your Stuff Lives

When a coroutine is called, the compiler allocates a **coroutine frame** on the heap. This is where everything lives while the coroutine is suspended.

COROUTINE FRAME (Heap)		
=====		
+-----+		
Promise Object	<-- Your "control	
(your promise_type lives here)	panel"	
+-----+		
+-----+		
Function Parameters (copies)	<-- Copied in at	
e.g., int n, string name	the start	
+-----+		
+-----+		
Local Variables	<-- Preserved	
e.g., int i = 0; double sum = 0;	across suspends	
+-----+		
+-----+		
Suspend Point / Resume Point	<-- Where to pick	
(basically a hidden state machine)	up next time	
+-----+		
+-----+		
coroutine_handle	Points back to	
(compiler manages this)	this frame	
+-----+		

This is why coroutines can suspend and resume — their entire state isn't on the stack (which would vanish), it's on the heap. They're like a save file in a video game.

The Required Pieces: An Anatomy Diagram

Here's everything involved and how it all connects:

YOUR COROUTINE FUNCTION

```
Generator<int> count_up_to(int n)
{
    for (int i=1; i<=n; ++i)
        co_yield i;
}
```

compiler sees `co_yield`
and looks up return type

YOUR RETURN TYPE

```
struct Generator<T> {
    struct promise_type { ... };
    coroutine_handle<promise_type> h;
    bool next();
    T value();
};
```

COMPILER-GENERATED CODE (simplified):

1. frame = new coroutine_frame
2. promise = frame->promise_type{}
3. return_obj = promise.get_return_object()
4. co_await promise.initial_suspend()
5. try { <YOUR BODY HERE> }
6. catch(...) { promise.unhandled_exception() }
7. co_await promise.final_suspend()
8. destroy frame

That compiler-generated wrapper is the most important diagram in this entire tutorial. Burn it into your brain. Let's zoom in.

The Compiler's Secret Transformation

When you write:

```
Generator<int> count_up_to(int max) {  
    for (int i = 1; i <= max; ++i) {  
        co_yield i;  
    }  
}
```

The compiler conceptually transforms it into something like this. You never write this code — but understanding it is the key to everything:

```
// PSEUDOCODE -- what the compiler does behind the scenes  
Generator<int> count_up_to(int max) {  
    // 1. Allocate the coroutine frame on the heap  
    auto* frame = new __coroutine_frame_count_up_to{};  
  
    // 2. Copy parameters into the frame  
    frame->max = max;  
  
    // 3. Construct the promise object inside the frame  
    auto& promise = frame->__promise;  
    // (promise_type is default-constructed)  
  
    // 4. Get the return object (this is what the CALLER receives)  
    Generator<int> return_object = promise.get_return_object();  
  
    // 5. Should we suspend before running the body?  
    co_await promise.initial_suspend();  
    // (if this returns suspend_always, the coroutine pauses HERE  
    // before executing a single line of your code!)  
  
    try {  
        // 6. ===== YOUR ACTUAL CODE RUNS HERE =====  
        for (int i = 1; i <= frame->max; ++i) {  
            // co_yield i; becomes:  
            co_await promise.yield_value(i);  
        }  
    }  
}
```

```

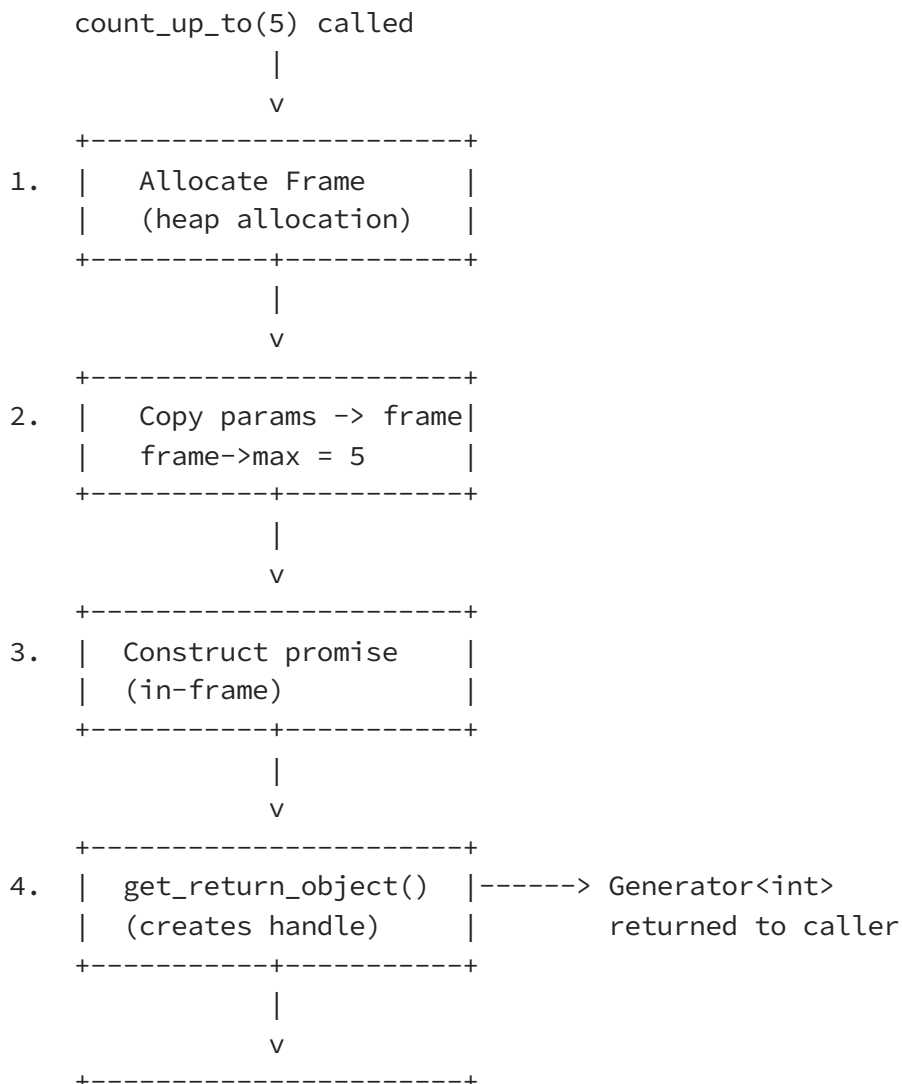
        // (this stores i somewhere in the promise,
        // then suspends the coroutine)
    }
    // implicit co_return;
    // which calls promise.return_void();
} catch (...) {
    // 7. If your code throws, the promise handles it
    promise.unhandled_exception();
}

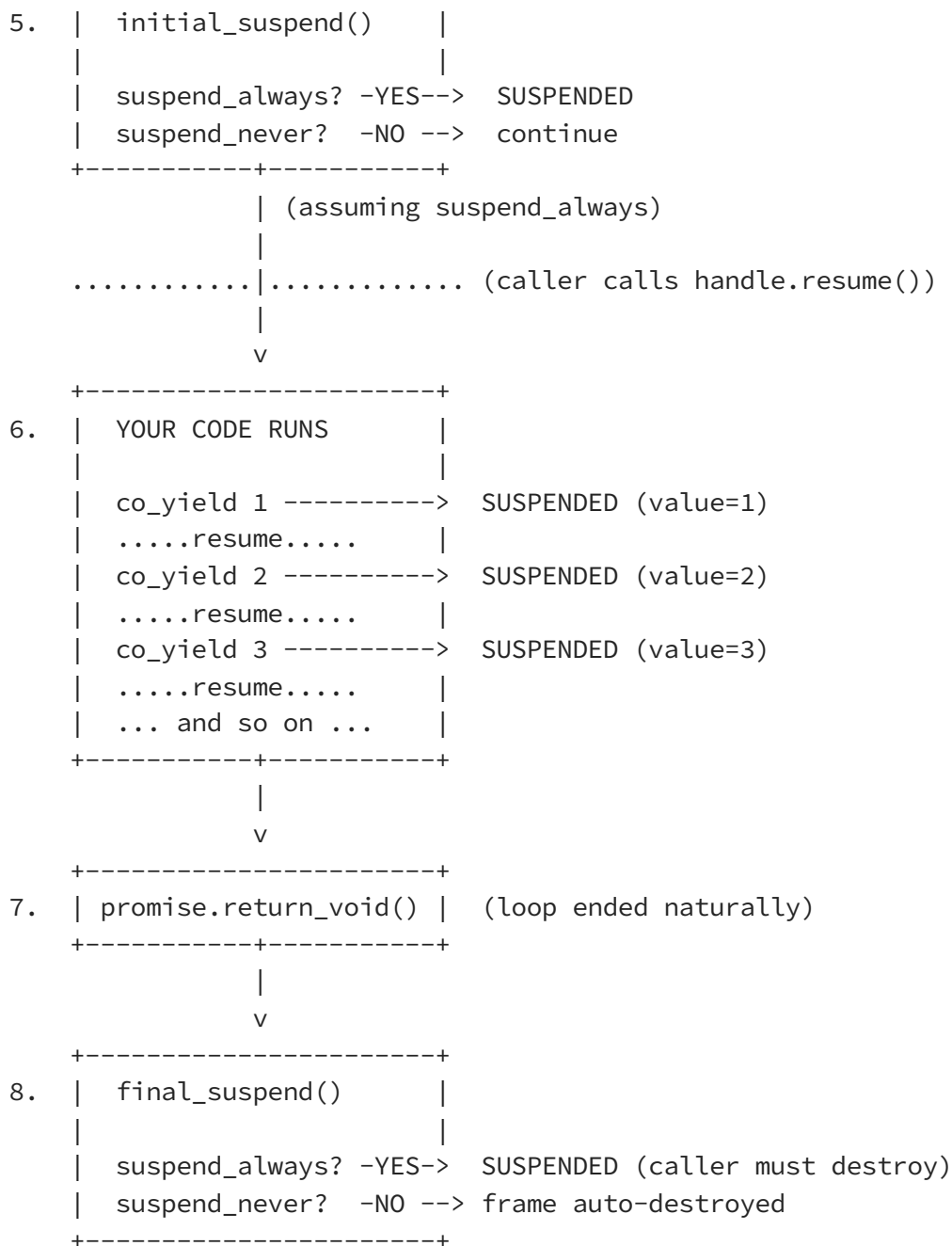
// 8. Should we suspend at the very end?
co_await promise.final_suspend();

// 9. If we didn't suspend at final_suspend,
//     destroy the frame now
// (Usually we DO suspend, and let the caller destroy it)
}

```

Let's visualize this flow:





Pro Tip: Almost always use `suspend_always` for `final_suspend()` . If you use `suspend_never` , the coroutine frame is destroyed immediately, and if anyone is still holding a `coroutine_handle` ... dangling handle. Undefined behavior. Tears.

The Promise Type: Your Control Panel Explained

The promise type has a bunch of methods, and each one is a “hook” that the compiler calls at a specific point. Here’s a cheat sheet:

PROMISE TYPE CHEAT SHEET	
METHOD	WHEN THE COMPILER CALLS IT
<code>get_return_object()</code>	Right after construction. Returns the thing the caller gets. (Your Generator, Task, etc.)
<code>initial_suspend()</code>	Before the body runs. <code>suspend_always</code> = lazy (don't start yet) <code>suspend_never</code> = eager (start immediately)
<code>final_suspend()</code>	After the body finishes (or throws). <code>suspend_always</code> = stay alive for cleanup <code>suspend_never</code> = self-destruct
<code>yield_value(val)</code>	When your code does: <code>co_yield val</code> ; Store the value, then suspend.
<code>return_void()</code>	When your code does: <code>co_return</code> ; (or falls off the end of the function)
<code>return_value(val)</code>	When your code does: <code>co_return val</code> ; (use THIS or <code>return_void</code> , never both!)
<code>unhandled_exception()</code>	When your code throws and doesn't catch. You can store it, rethrow, or terminate.

How the Compiler Finds Your Promise Type ?

```

Generator<int> my_coroutine(int x, float y) {
    co_yield 42;
}

|
| compiler sees co_yield --> "This is a coroutine!"
|
v

```

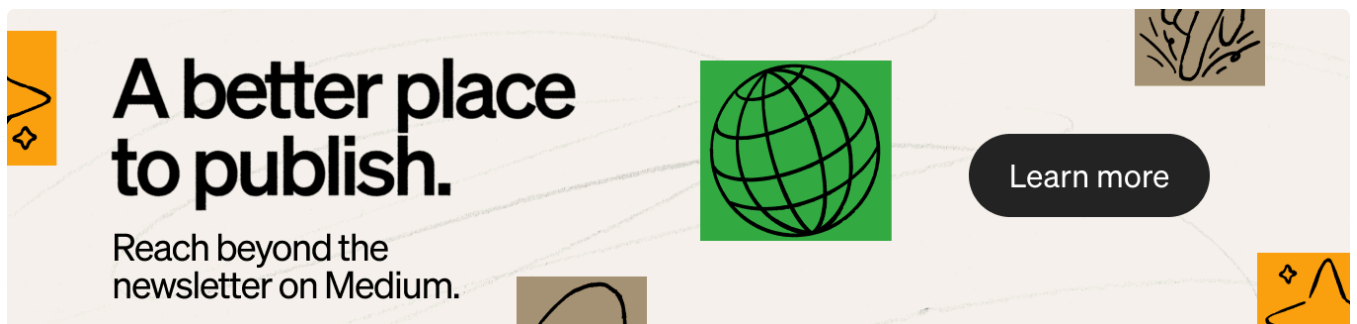
```

| What's the return type? --> Generator<int>
|
|   v
| Look for: Generator<int>::promise_type
|
|       (If not found, the compiler also tries
|       std::coroutine_traits<Generator<int>, int, float>
|       ::promise_type -- but the nested type is simpler)
|
|   v
| Found it! Use Generator<int>::promise_type for everything.
|
+-----+

```

Now Let's Build Our First Coroutine: A Generator

Enough theory. Let's build a **generator** — a coroutine that produces a sequence of values lazily (one at a time, on demand). This is the “Hello World” of coroutines.



Here's what we want to write:

```

Generator<int> count_up_to(int max) {
    for (int i = 1; i <= max; ++i) {
        co_yield i; // "Here's a value. Now pause me."
    }
    // Implicit co_return at the end
}

int main() {
    auto gen = count_up_to(5);
    while (gen.next()) {
        std::cout << gen.value() << "\n"; // Prints 1, 2, 3, 4, 5
    }
}

```

Clean. Elegant. But the compiler sees `co_yield` and immediately demands:

“Where’s the `promise_type`? Where's the return object? WHERE IS THE PAPERWORK?!”

So let’s write the paperwork. I mean, the `Generator` class.

The Complete Generator Implementation:

```
#include <coroutine>
#include <iostream>
#include <exception>

template <typename T>
class Generator {
public:
    // =====
    // THE PROMISE TYPE -- The "control panel" for the coroutine
    // =====
    struct promise_type {
        T current_value; // Stores the most recently yielded value

        // What to return to the caller when the coroutine is created
        Generator get_return_object() {
            return Generator{
                std::coroutine_handle<promise_type>::from_promise(*this)
            };
        }

        // Suspend immediately -- don't run the body until asked
        std::suspend_always initial_suspend() noexcept { return {}; }

        // Suspend at the end -- let the Generator destructor clean up
        std::suspend_always final_suspend() noexcept { return {}; }

        // When the coroutine does: co_yield value;
        std::suspend_always yield_value(T value) {
            current_value = std::move(value);
            return {};
        }

        // Our generator doesn't return a final value, just yields
        void return_void() {}

        // If something throws inside the coroutine body
        void unhandled_exception() {
            std::terminate(); // Nuclear option. You can rethrow instead.
        }
    };
};
```

```

};

// =====
//  THE GENERATOR CLASS -- What the caller interacts with
//  =====

explicit Generator(std::coroutine_handle<promise_type> handle)
    : handle_(handle) {}

~Generator() {
    if (handle_) {
        handle_.destroy();
    }
}

// No copying! The handle is a unique resource.
Generator(const Generator&) = delete;
Generator& operator=(const Generator&) = delete;

// Move is fine though
Generator(Generator&& other) noexcept
    : handle_(std::exchange(other.handle_, nullptr)) {}

Generator& operator=(Generator&& other) noexcept {
    if (this != &other) {
        if (handle_) handle_.destroy();
        handle_ = std::exchange(other.handle_, nullptr);
    }
    return *this;
}

// Advance to the next value. Returns false if done.
bool next() {
    handle_.resume();
    return !handle_.done();
}

// Get the current yielded value
T value() const {
    return handle_.promise().current_value;
}

private:
    std::coroutine_handle<promise_type> handle_;
};

```

And now, the coroutine itself:

```

Generator<int> count_up_to(int max) {
    for (int i = 1; i <= max; ++i) {
        co_yield i;
    }
}

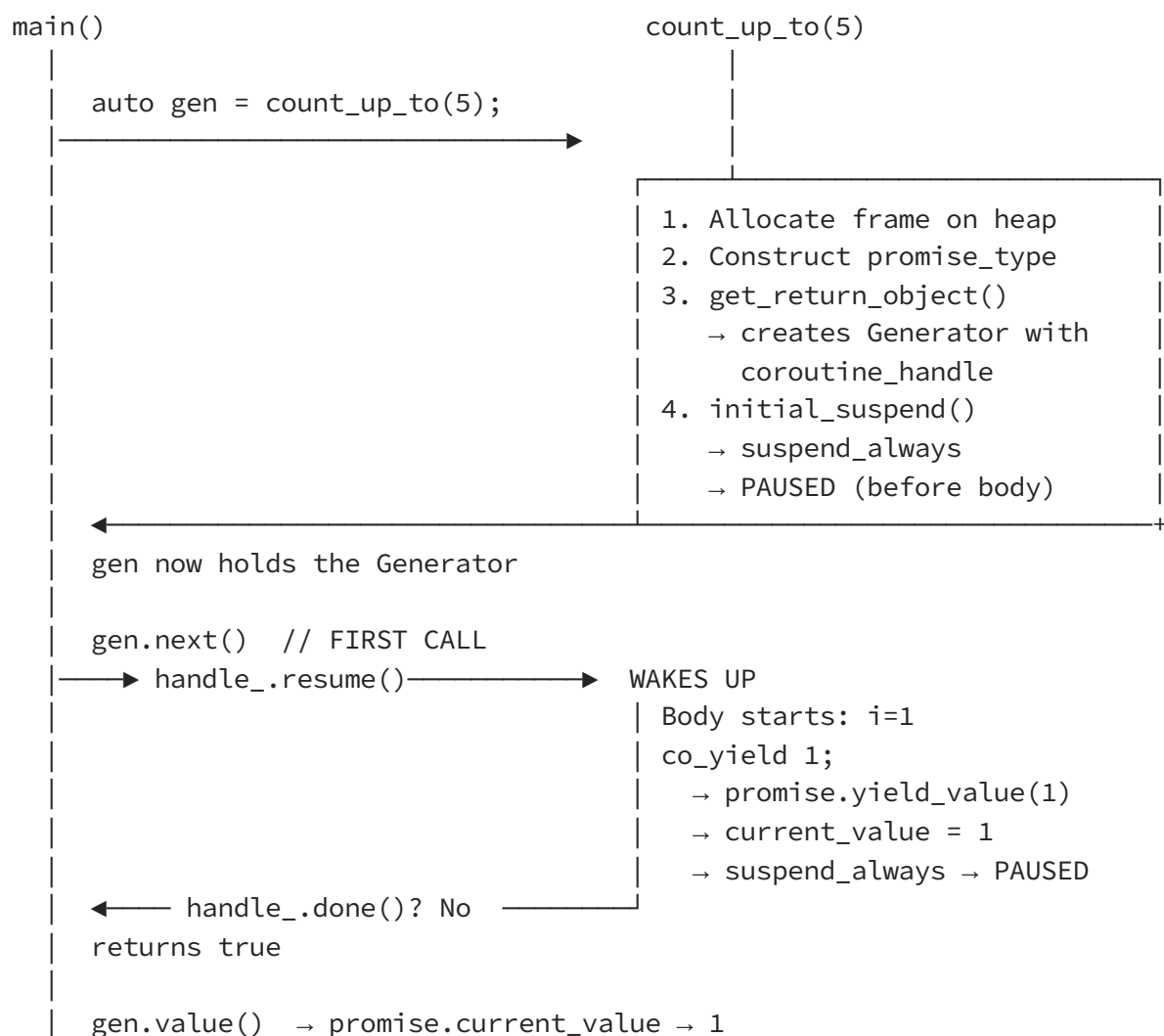
int main() {
    auto gen = count_up_to(5);

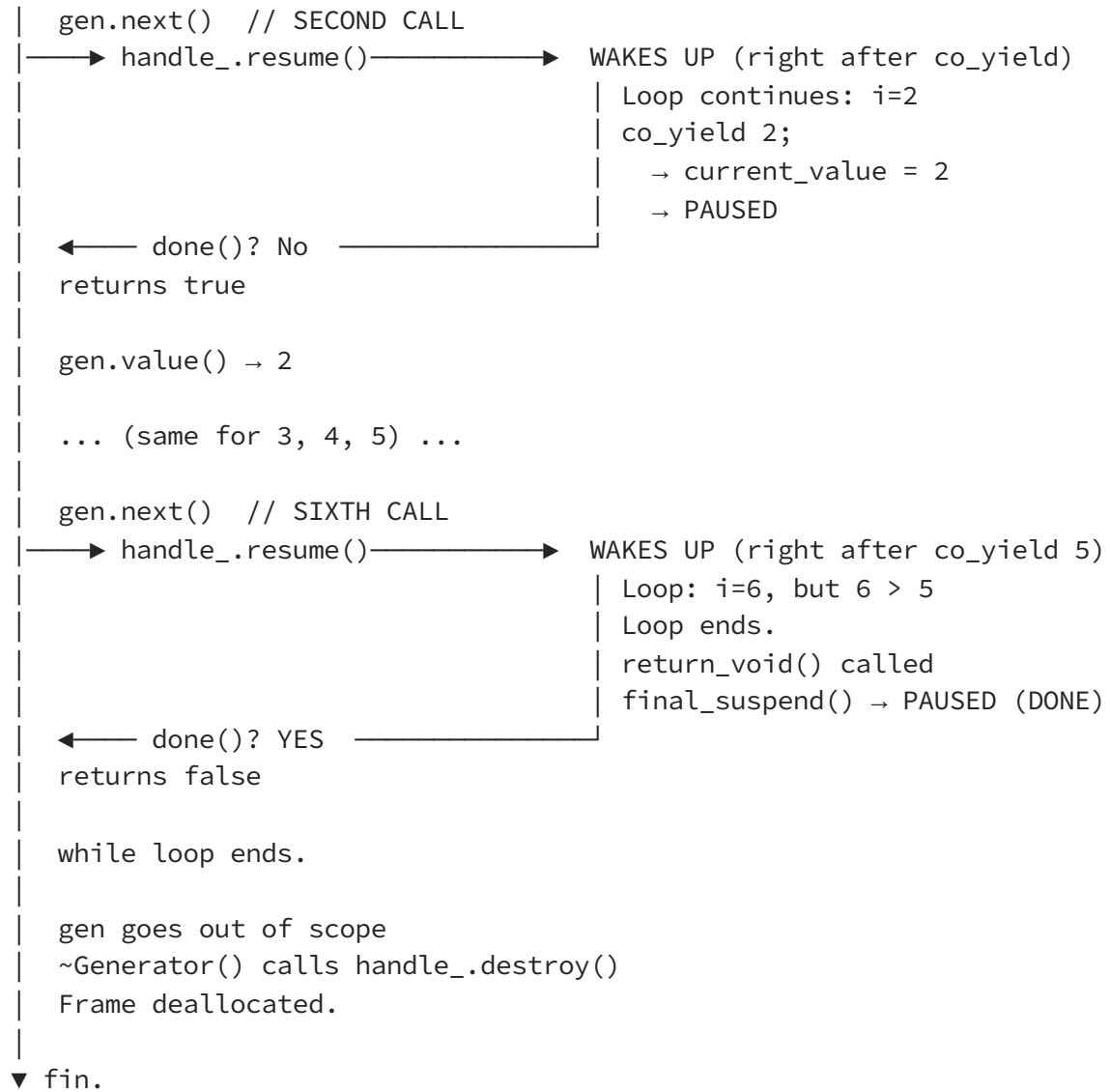
    while (gen.next()) {
        std::cout << gen.value() << "\n";
    }
    // Output: 1 2 3 4 5 (each on its own line)
}

```

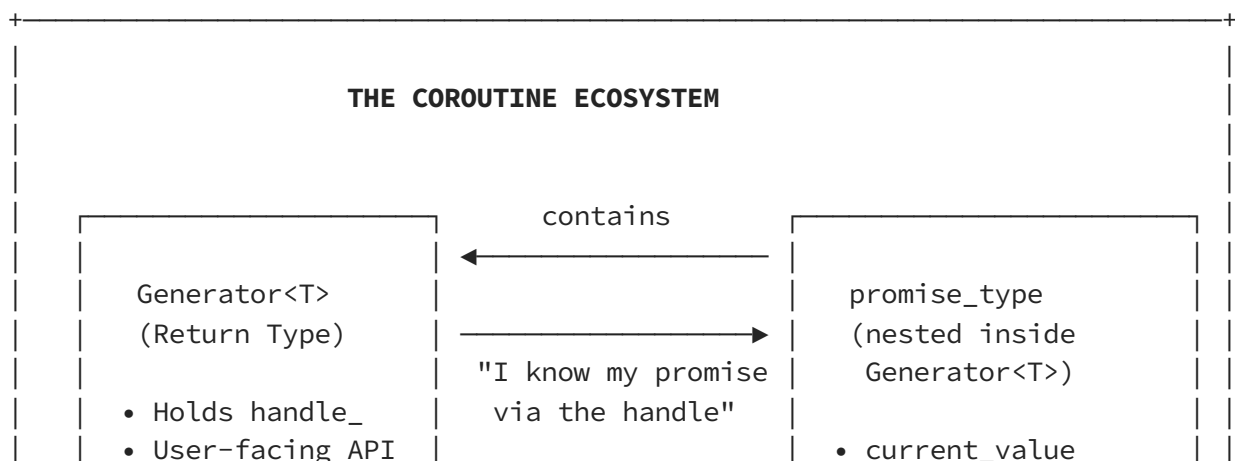
That's it. That actually works. Compile with `-std=c++20` and you're golden.

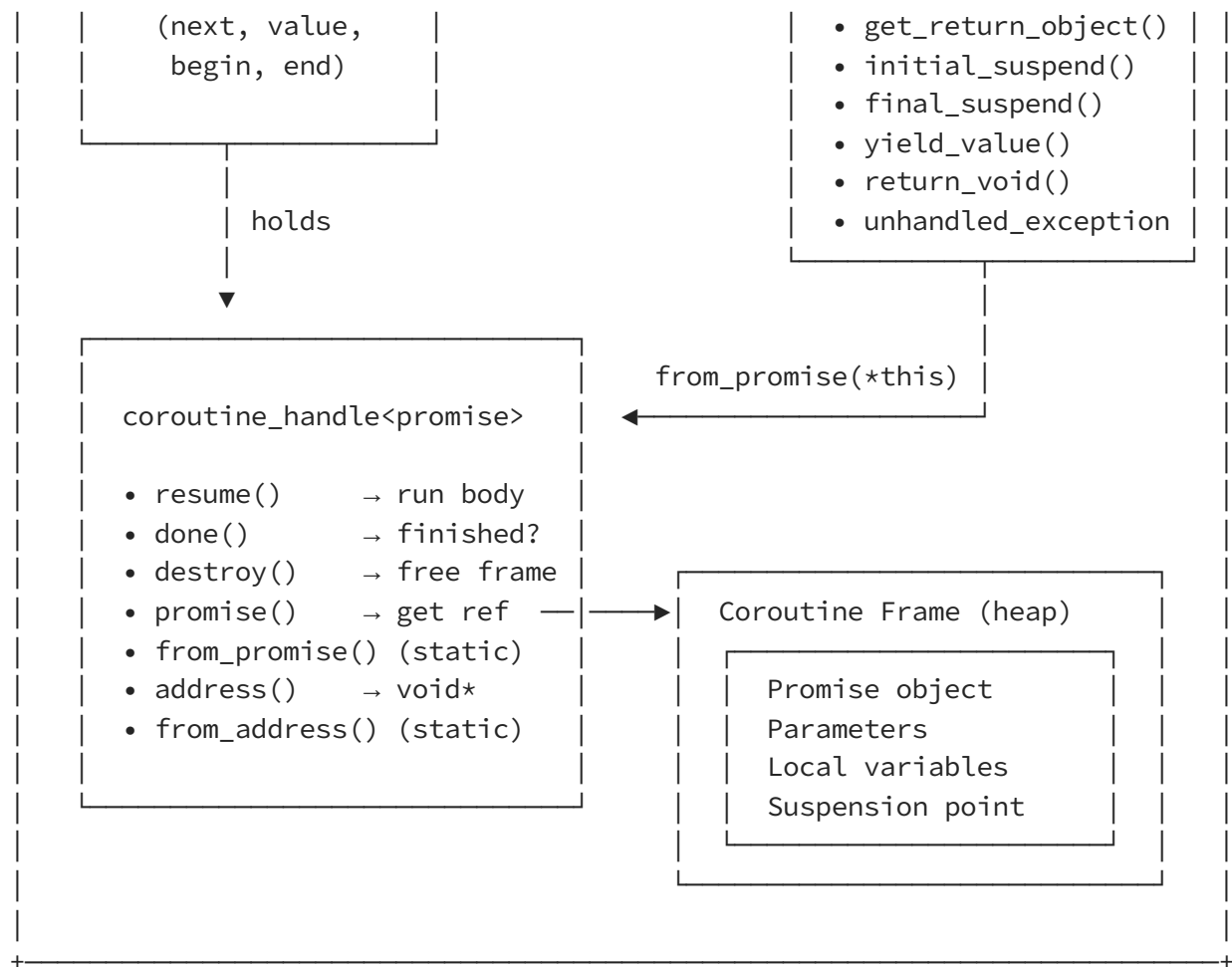
Let's Trace What Happens Step by Step:



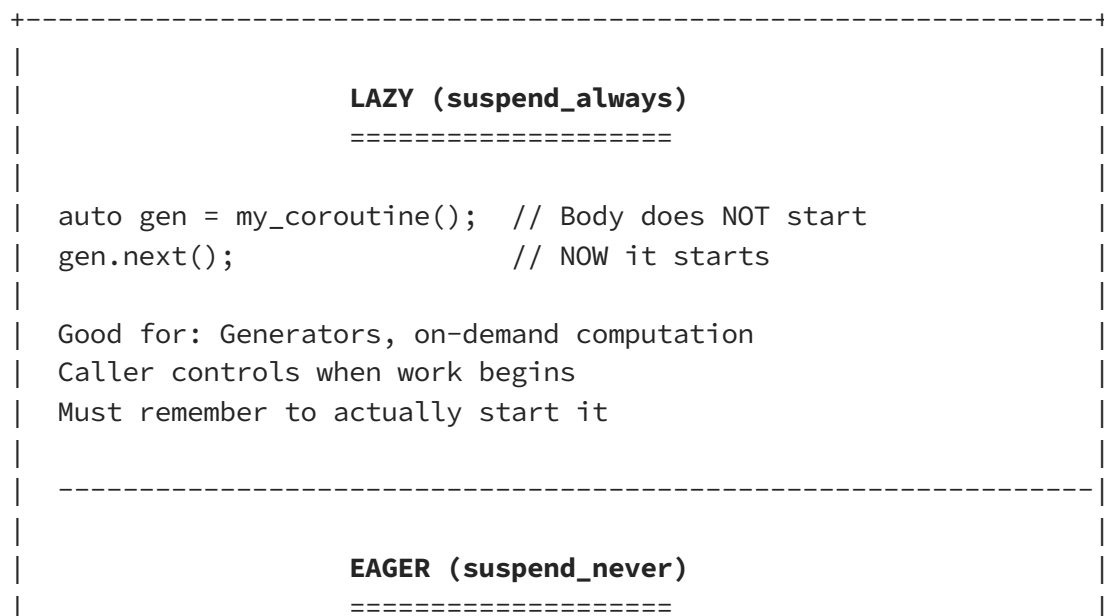


The Relationship Between All The Types





Lazy vs. Eager: The `initial_suspend` Decision



```
| auto task = my_coroutine(); // Body starts IMMEDIATELY  
|                               // runs until first co_await  
|
```

```
| Good for: Tasks, async operations  
| Work starts right away  
| Less control over when execution begins  
|
```

```
+-----+
```

Compiling and Running

Make sure you're using a C++20-compliant compiler:

```
# GCC (10+)  
g++ -std=c++20 -fcoroutines -o coroutines main.cpp  
  
# Clang (14+)  
clang++ -std=c++20 -stdlib=libc++ -o coroutines main.cpp  
  
# MSVC (Visual Studio 2019 16.8+)  
cl /std:c++20 main.cpp
```

Note: Older versions of GCC/Clang may need the `-fcoroutines` flag. On modern compilers (2023+), just `-std=c++20` should be enough.

Part 1 Checkpoint

Let's make sure you've got the foundations locked in before we proceed:

```
+-----+  
|  
| [x] A coroutine SUSPENDS and RESUMES  
|  
| [x] co_await, co_yield, co_return make a function a coroutine  
|  
| [x] The coroutine frame lives on the HEAP  
|
```

- [x] You must define a return type with a nested `promise_type`
- [x] The compiler generates the wrapper code that calls your `promise_type` methods at the right times
- [x] `coroutine_handle` is your remote control for the coroutine
- [x] You can build and use a working `Generator<T>!`

**If any of these feel shaky, re-read the sections above.
Part 2 builds directly on ALL of these concepts.**

Go grab a coffee, stretch those legs, and when you're ready — continue with **Part 2(Advanced C++20 Coroutines: Mastering `co_await` and `Task`)**.

C Plus Plus Language

Cpp20

Software Development

Asynchronous Programming

Programming



Follow

Published in Towards Dev

9.8K followers · Last published 18 hours ago

A publication for sharing projects, ideas, codes, and new theories.



Following ▾

Written by Sagar

91 followers · 4 following

System Programmer | C++ | Linux System Programming | Rust | Python | Cloud Computing | DevOps | Low Latency Backends